0. Orientation: Programs, Computers, and Go

00. Orientation: Programs, Computers, and Go

Phase: Phase 0 — Orientation
Estimated time: 1 day
Slug: `00-orientation`

Learning objectives

- Explain what a program is and how source code becomes running software
- Describe CPU, memory, and the OS at a beginner level
- Contrast Go with Python (compiled, static types, concurrency model)
- Install the Go toolchain and run your first program
- Adopt the study method: attempt → struggle → solution → rewrite from memory

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Before writing production Go you need a mental model of how source becomes a running process, and how Go's toolchain differs from Python. Orientation prevents weeks of cargo-cult commands.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- A program is source text turned into CPU instructions; the OS schedules it as a process.
- Go compiles to native binaries; `go run` is compile+link+execute for development.
- Modules (`go.mod`) pin dependency versions; packages group related files.
- The study loop (attempt → struggle → solution → rewrite) builds retrieval, not recognition.
- `GOPATH` is legacy for most work; modules are the default since Go 1.14+.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-orientation: && cd /tmp/gw-orientation:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Goodway")
    fmt.Printf("2+2=%d\n", 2+2)
}
```

Run it:

```
go run .
```

Expected output:

```
Hello, Goodway
2+2=4
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Install Go and run `go version`.
2. `mkdir -p /tmp/hello && cd /tmp/hello && go mod init example.com/hello`
3. Create `main.go` with the example; run `go run .`
4. `go build -o hello && ./hello` and confirm the same output.
5. Run `go env GOROOT GOPATH GO111MODULE` and skim what each means.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

What is a program?

A **program** is a precise sequence of instructions a computer can execute. You write **source code** (human-readable text). A **compiler** or **interpreter** turns that into machine instructions the CPU understands.

- **CPU**: executes instructions one at a time (very fast).
- **Memory (RAM)**: temporary storage for values your program uses while running.
- **Disk**: durable storage (files, databases).
- **OS**: manages processes, files, networking; your Go program runs as a process.

Compiled vs interpreted (Go vs Python)

	Python	Go
How it runs	Interpreter / VM	Compiled to native machine code
Types	Dynamic (checked at runtime)	Static (checked at compile time)
Speed	Often slower for CPU work	Typically fast
Concurrency	GIL limits CPU threads	Goroutines + scheduler

Go catches many mistakes **before** your program runs. That feels stricter than Python at first — and that strictness is a feature for infrastructure work.

The Go toolchain

After installing Go (https://go.dev/dl/):

```
go version          # confirm install
go env GOPATH GOROOT
mkdir -p hello && cd hello
go mod init example.com/hello
```

Create `main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Goodway")
}
```

```
go run .            # compile + run
go build -o hello   # produce a binary
./hello
go test ./...       # run tests (later lessons)
```

How to study this course

1. Read the lesson PDF carefully; type examples yourself (do not copy-paste blindly).
2. Attempt every exercise **without** opening `solutions/`.
3. If stuck >20 minutes, re-read the concept section, then try a smaller version.
4. Only then compare with the solution — write notes on *why* it works.
5. Next day: rewrite the solution from memory without looking.

Track progress in `curriculum/PROGRESS.md`.

Failure modes to watch while coding

- Expecting Go to feel like Python (dynamic typing, exceptions everywhere)
- Skipping the rewrite-from-memory step — that is where mastery forms
- Installing multiple Go versions without understanding modules

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python: `python hello.py` runs a script. Go: you usually build a package/module.
- Python catches many type errors at runtime; Go rejects them at compile time.
- There is no REPL-as-primary workflow — use `go test` and small `main` programs.
- Virtualenv ≠ modules: Go modules version *dependencies*, not the language runtime.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Keep `curriculum/PROGRESS.md` updated. Tomorrow, recreate hello without looking at today's files.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Orientation: Programs, Computers, and Go" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Expecting Go to feel like Python (dynamic typing, exceptions everywhere)
- Skipping the rewrite-from-memory step — that is where mastery forms
- Installing multiple Go versions without understanding modules

Exercises

Work in `exercises/00-orientation/`. Read that folder's `README.md` before you code.

What you will build

Implement `Greet(name string) string` so it returns `Hello, <name>`. This drills package layout, exported funcs, and `go test` as feedback.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/00-orientation/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does the Go compiler do that a Python interpreter does differently?
2. Name three commands you will use daily (`go run`, `go test`, …).
3. Why does this course ban looking at solutions first?

Further reading

- https://go.dev/doc/install
- https://go.dev/tour/